面向对象分析与设计

Object Oriented Analysis and Design

——概要设计

Architectural Design

邱明 博士

厦门大学信息学院

mingqiu@xmu.edu.cn

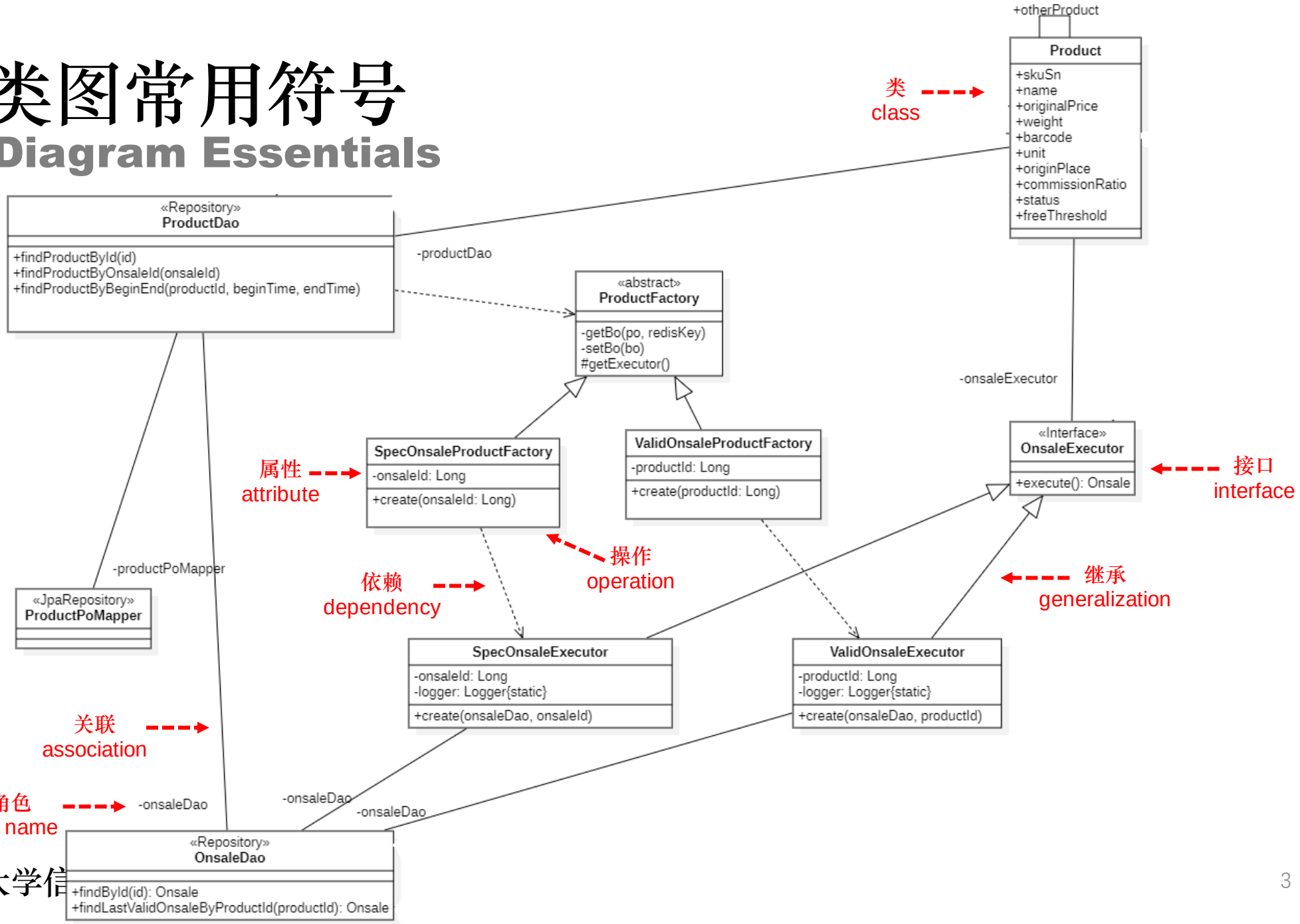
2025年秋季学期

1. UML类图和包图

UML Class Diagram and Package Diagram

1.1 类图常用符号

Class Diagram Essentials



1.1 类图常用符号
Class Diagram Essentials

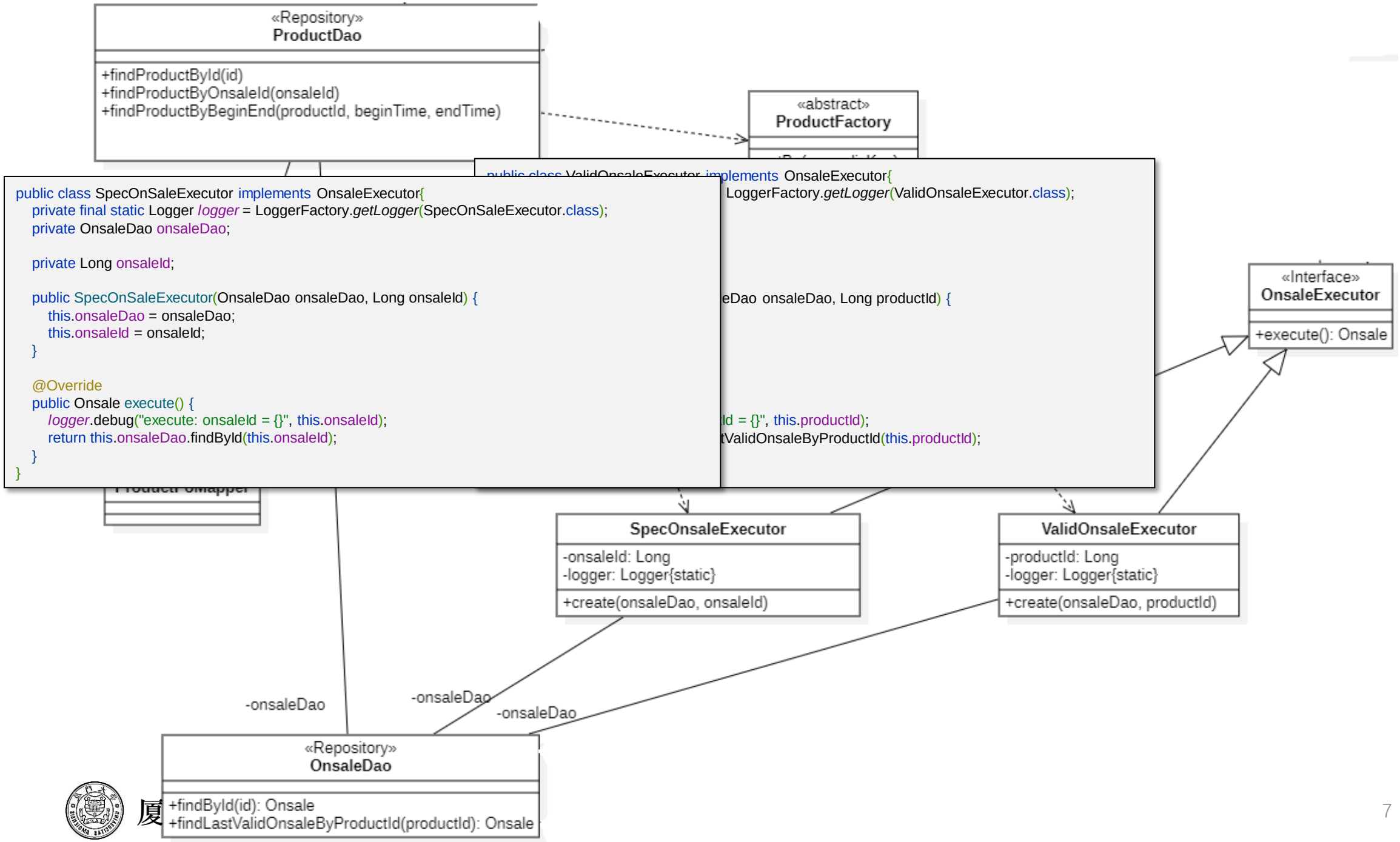
- 属性（Attributes）
  - 类的原始数据类型属性
  - visibility name : type multiplicity = default {property-string}
  - - onsaleId: Long [1] = null {auto-incr}
  - Long onsaleId;

1.1 类图常用符号
Class Diagram Essentials

- 操作（Operation）
  - visibility name (parameter-list) : return-type {property-string}
  - + getPlayer(name : String) : Player {exception IOException}
  - public Player getPlayer(String name) throws IOException;
  - Create用于描述创建对象（new）
  - get/set方法可以省略

1.2 元素间的关系
Relationship

- 关联（Association）
  - 自定义的类型的属性
- 继承（Generalization）
  - 元素之间的继承关系
- 依赖关系（Dependency）
  - 一个对象的修改会影响另一个对象

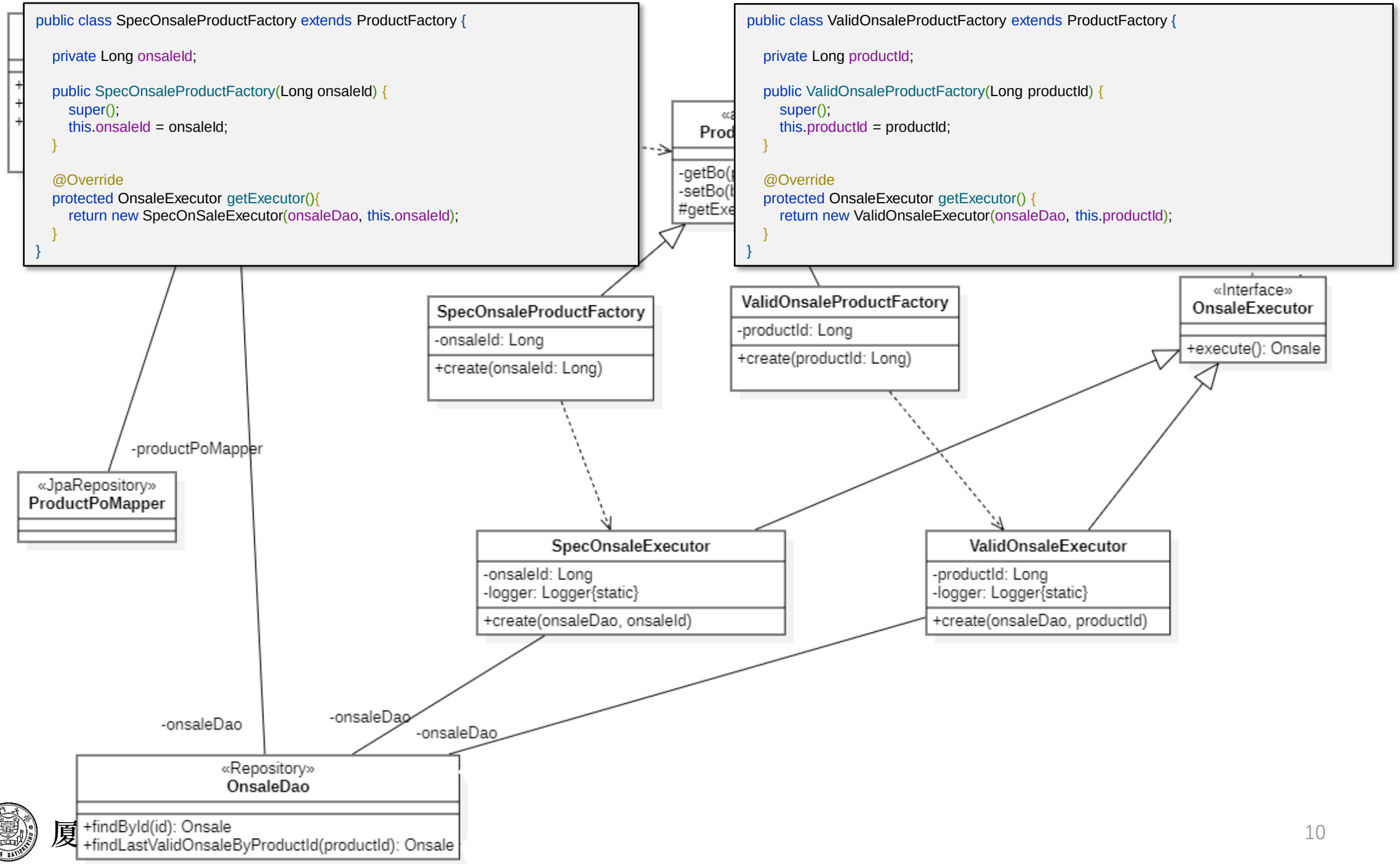

«Repository»
ProductDao
+findProductById(id)
+findProductByOnsaleId(onsaleId)
+findProductByBeginEnd(productId, beginTime, endTime)
«abstract»
ProductFactory
«Interface»
OnsaleExecutor
+execute(): Onsale
SpecOnsaleExecutor
-onsaleId: Long
-logger: Logger{static}
+create(onsaleDao, onsaleId)
ValidOnsaleExecutor
-productId: Long
-logger: Logger{static}
+create(onsaleDao, productId)
-onsaleDao
-onsaleDao
-onsaleDao
«Repository»
OnsaleDao
+findById(id): Onsale
+findLastValidOnsaleByProductId(productId): Onsale
public class SpecOnSaleExecutor implements OnsaleExecutor{
private final static Logger logger = LoggerFactory.getLogger(SpecOnSaleExecutor.class);
private OnsaleDao onsaleDao;
private Long onsaleId;
public SpecOnSaleExecutor(OnsaleDao onsaleDao, Long onsaleId) {
this.onsaleDao = onsaleDao;
this.onsaleId = onsaleId;
}
@Override
public Onsale execute() {
logger.debug("execute: onsaleId = {}", this.onsaleId);
return this.onsaleDao.findById(this.onsaleId);
}
}
LoggerFactory.getLogger(ValidOnsaleExecutor.class);
eDao onsaleDao, Long productId) {
ld = {}", this.productId);
tValidOnsaleByProductId(this.productId);

1.3 泛型和标签
Stereotype and tag

- 泛型（Stereotype）
  - {« »}.对现有UML元素的重新定义
- 标签 tag
  - 用于描述UML元素的性质 {name1=value1, name2=value2}
- 抽象类和方法用 {abstract}tag表示
- Final类用{leaf} tag

1.4 四种依赖关系
Four Kinds of Dependency

- 依赖关系
  - 全局变量
  - 方法的参数
  - 局域变量
  - 静态方法

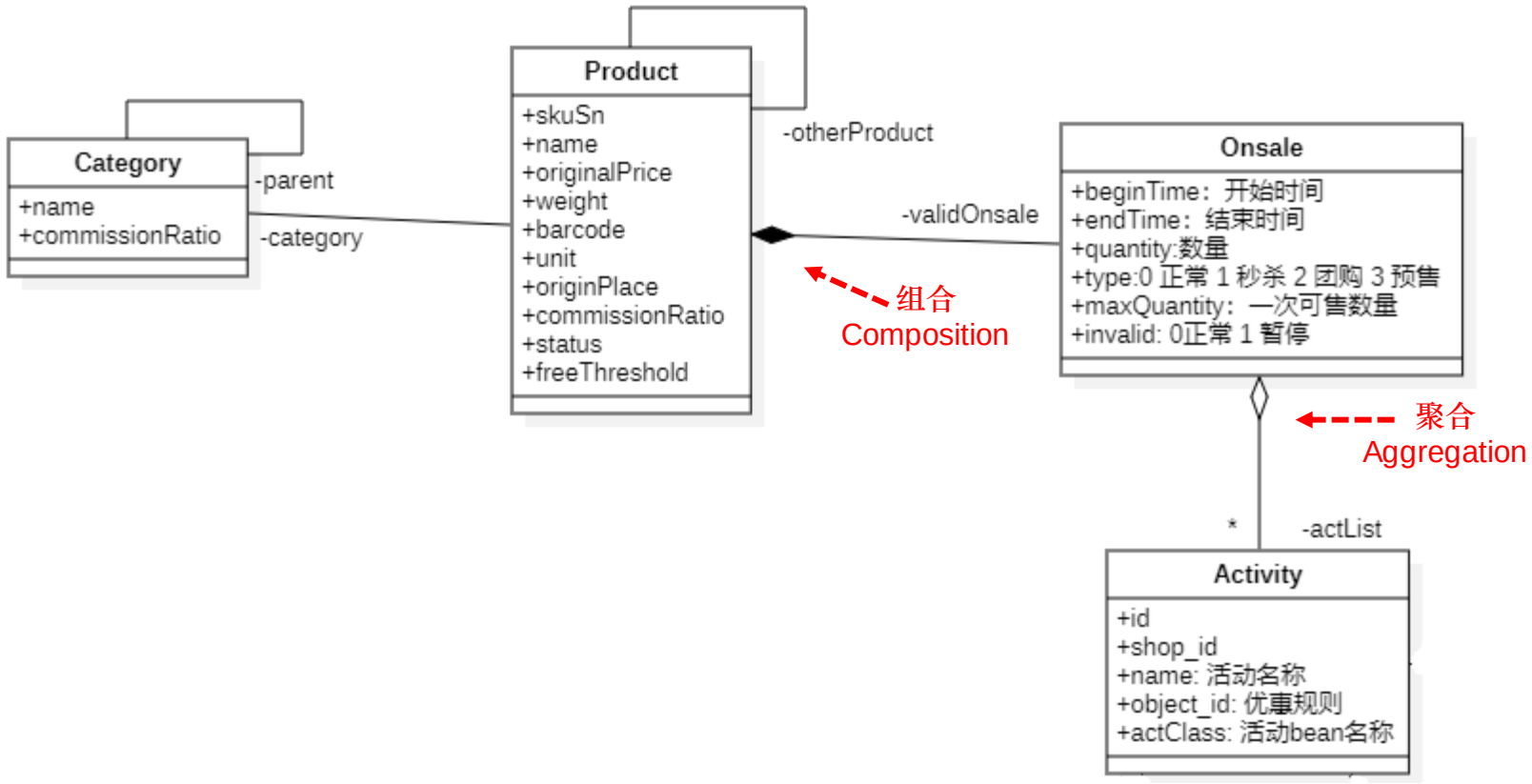

public class SpecOnsaleProductFactory extends ProductFactory {
private Long onsaleId;
public SpecOnsaleProductFactory(Long onsaleId) {
super();
this.onsaleId = onsaleId;
}
@Override
protected OnsaleExecutor getExecutor(){
return new SpecOnSaleExecutor(onsaleDao, this.onsaleId);
}
}
public class ValidOnsaleProductFactory extends ProductFactory {
private Long productId;
public ValidOnsaleProductFactory(Long productId) {
super();
this.productId = productId;
}
@Override
protected OnsaleExecutor getExecutor() {
return new ValidOnsaleExecutor(onsaleDao, this.productId);
}
}
SpecOnsaleProductFactory
-onsaleId: Long
+create(onsaleId: Long)
ValidOnsaleProductFactory
-productId: Long
+create(productId: Long)
«Interface»
OnsaleExecutor
+execute(): Onsale
-productPoMapper
«JpaRepository»
ProductPoMapper
SpecOnsaleExecutor
-onsaleId: Long
-logger: Logger{static}
+create(onsaleDao, onsaleId)
ValidOnsaleExecutor
-productId: Long
-logger: Logger{static}
+create(onsaleDao, productId)
-onsaleDao
-onsaleDao
-onsaleDao
«Repository»
OnsaleDao
+findById(id): Onsale
+findLastValidOnsaleByProductId(productId): Onsale

1.5 聚合和组合
Aggregation and Composition

- 聚合 （Aggregation）
  - 普通的一对多的关联关系.
- 组合 （Composition）
  - 整体和局部的关系
  - 局部对象只能属于一个母体对象
  - 局部对象必属于母体对象
  - 母体对象负责创建和销毁局部对象

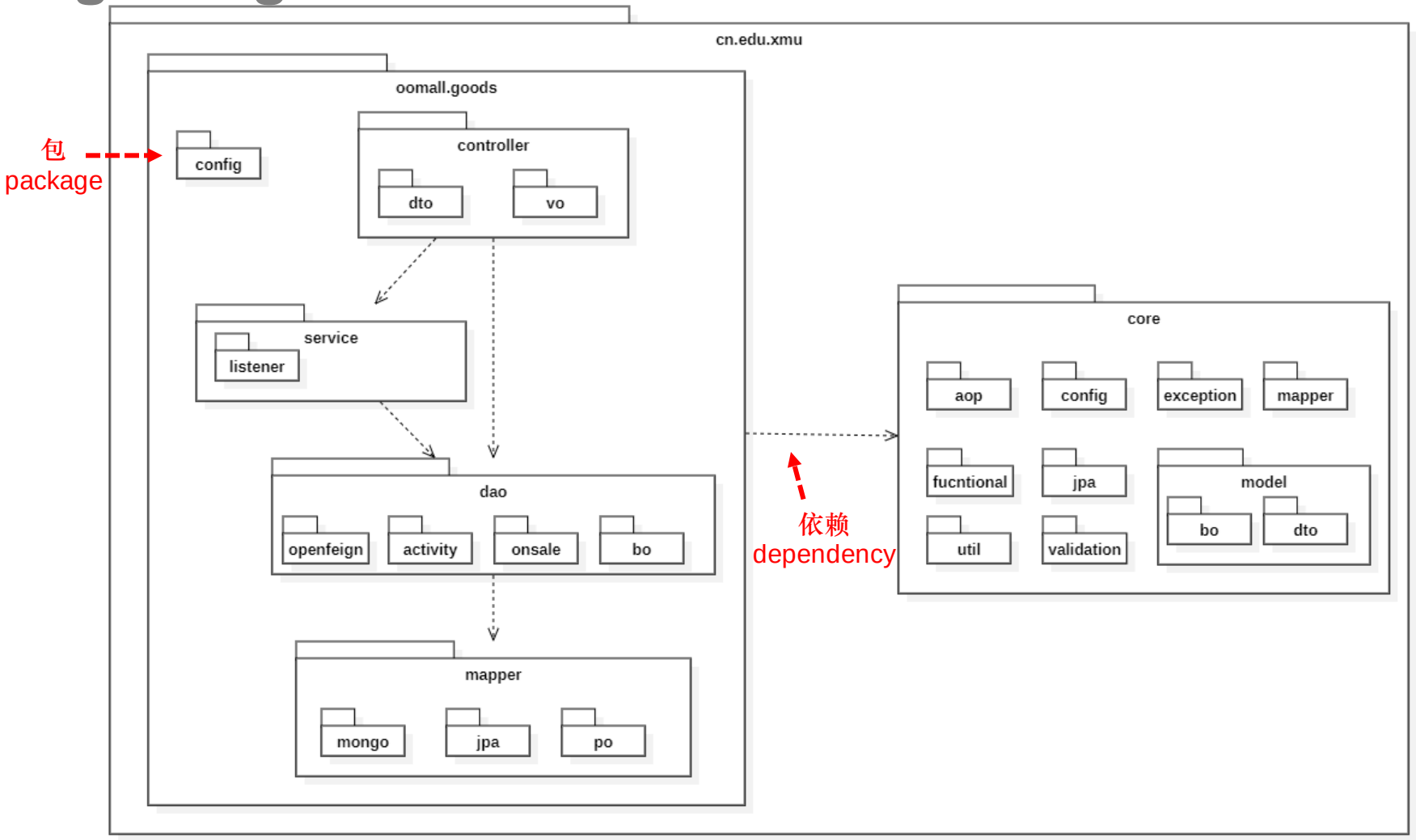
1.5 聚合和组合

Aggregation and Composition



1.6 包图常用符号

Package Diagram Essentials



2. 领域建模

Domain Modeling

2.1 领域模型
Domain Model

- 描述需求中存在的概念以及概念之间的关系，也被称为概念模型（Conceptual Model）
- 用UML类图描述需求中的概念和概念之间的关联
  - 也称可视字典（Visual Dictionary）

2.1 领域模型
Domain Model

- 描述需求中存在的概念以及概念之间的关系，也被称为概念模型（Conceptual Model）
- 以类图描述以下内容
  - 领域对象或概念类
  - 概念之间的关联
  - 概念的属性

2.1 领域模型
Domain Model

- 领域中的概念



符号（Symbol）

Product是电子商城销售的商品，它有名称、产地、价格、重量、条码和单位等属性。

内涵（Intension）



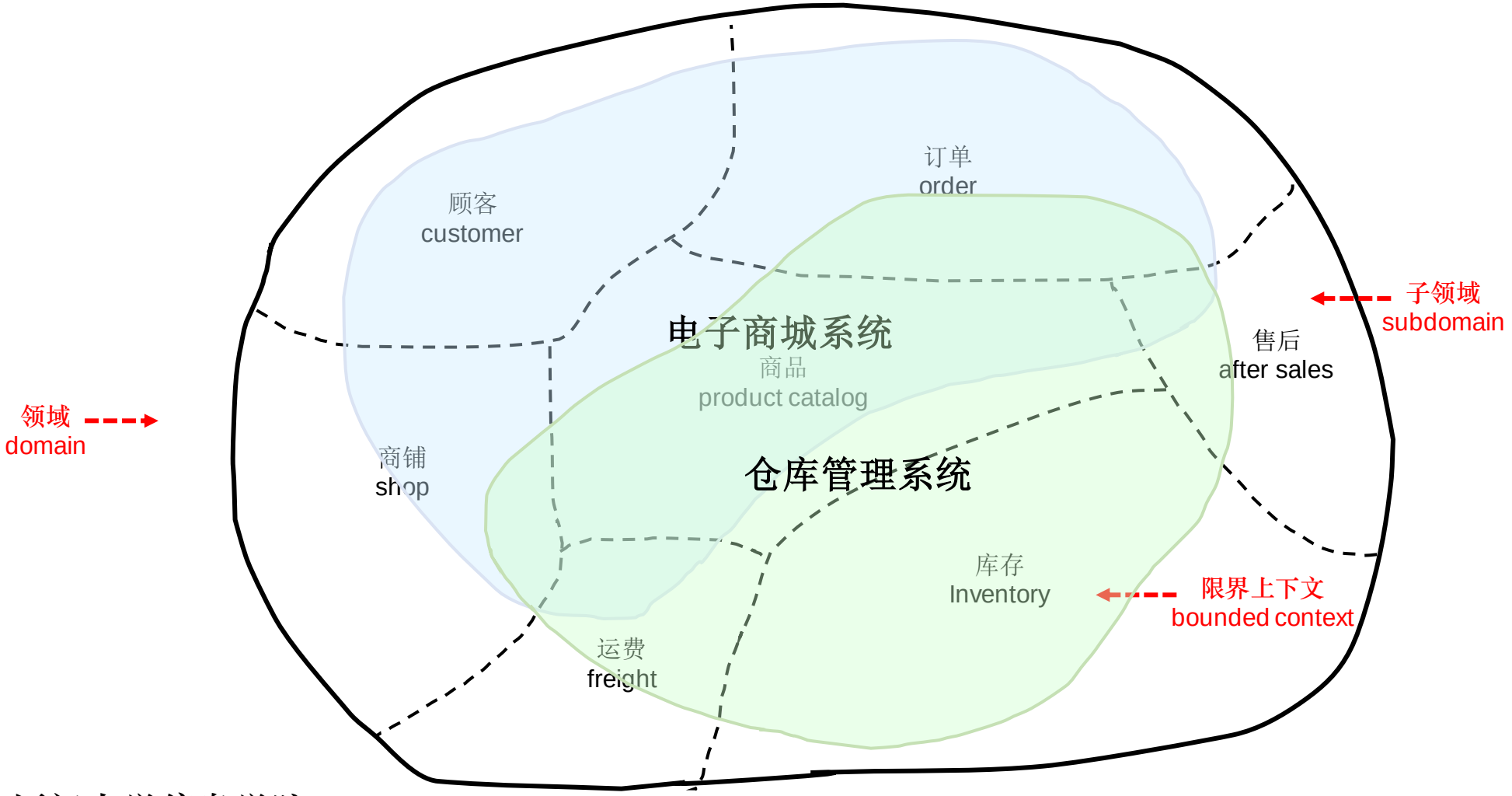
外延（Extension）

2.1 领域模型
Domain Model

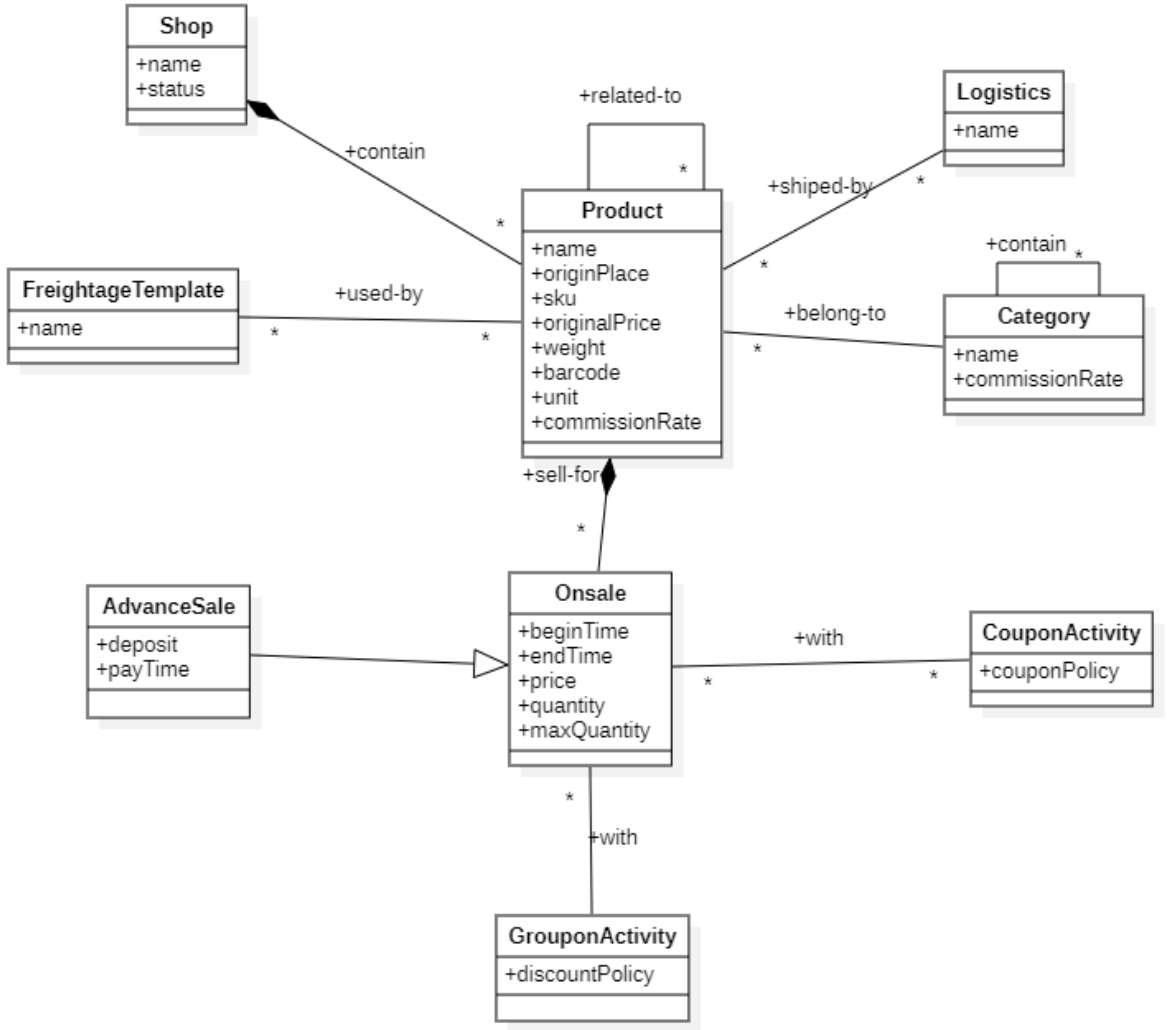
- 领域模型的作用
  - 理解需求
  - 发现需求背后的逻辑关系
  - 是对象模型的中间产品

2.2 领域、子领域和限界上下文

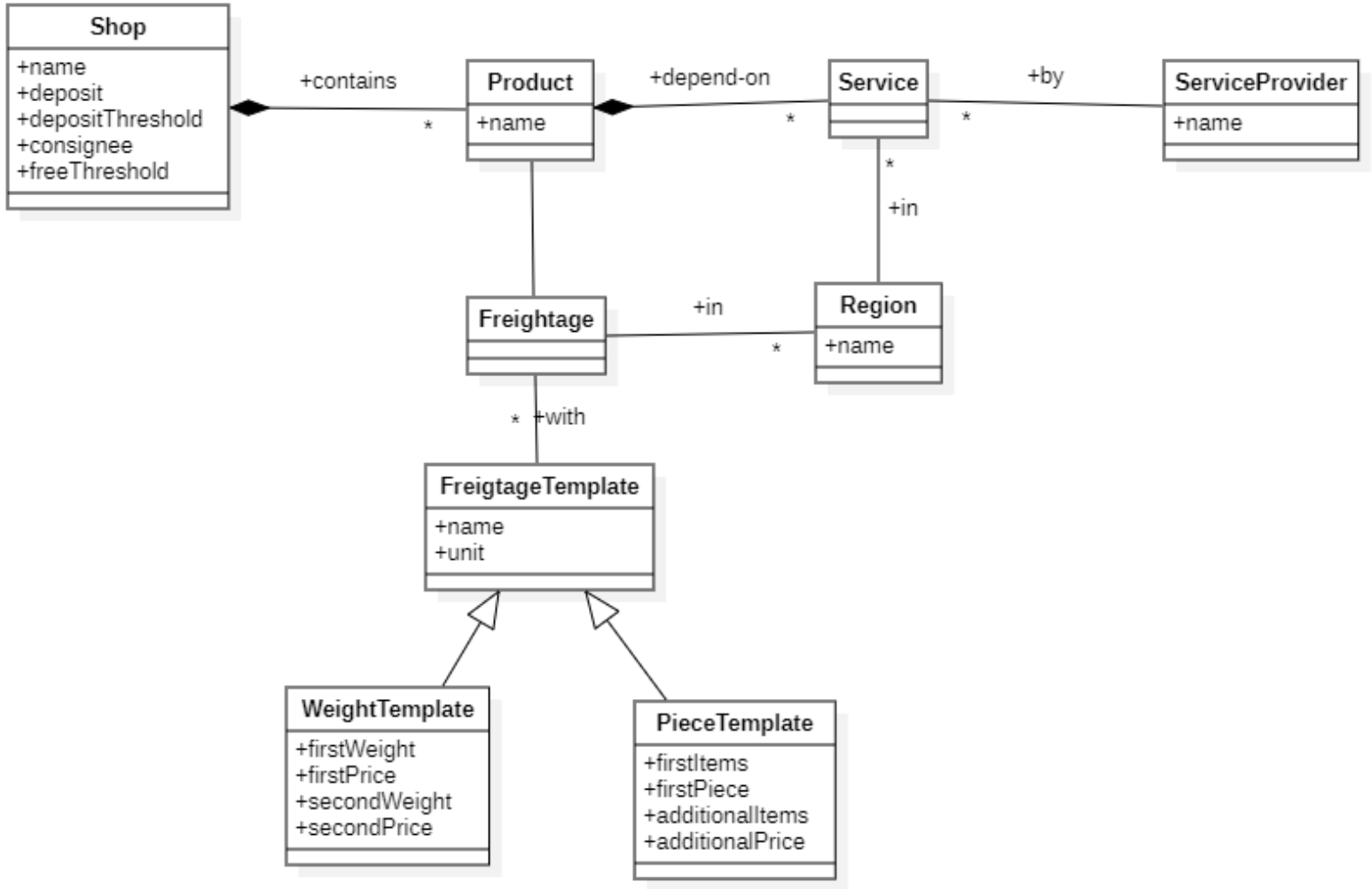
Domain, Subdomain and Bounded Context



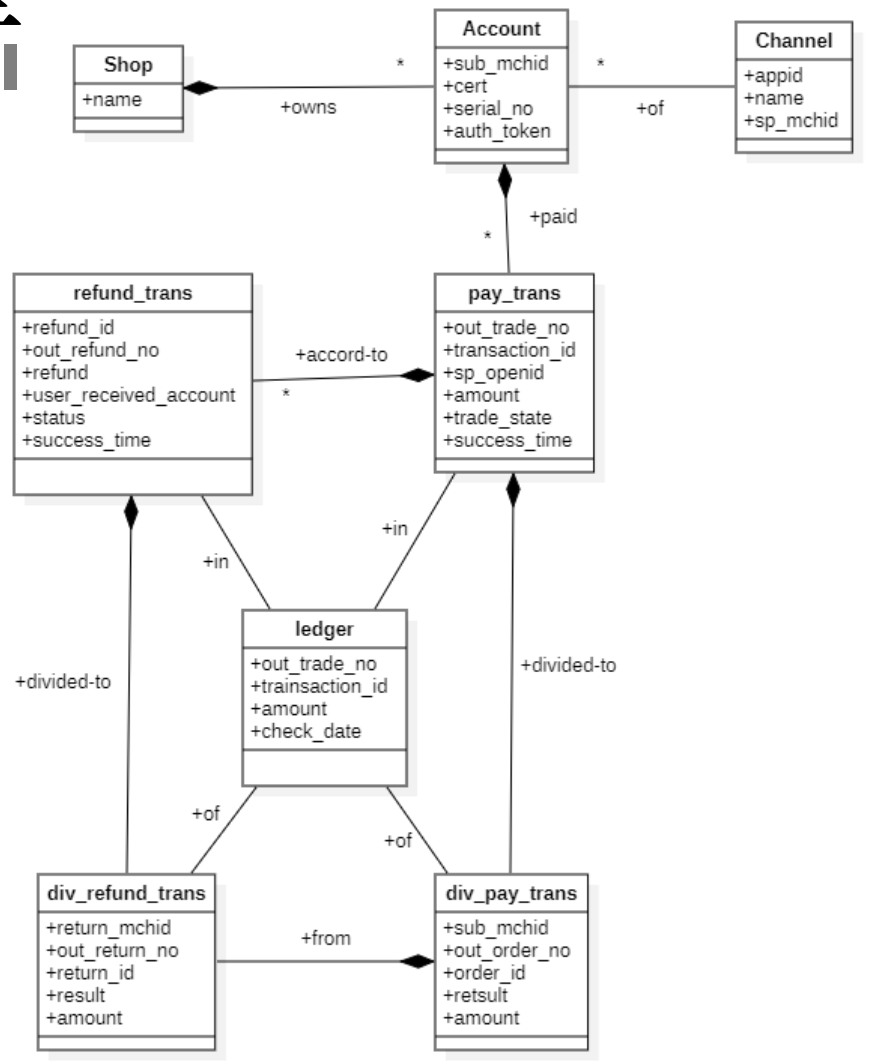
2.3 产品子领域模型
Product Subdomain Model



2.4 商铺子领域模型
Shop Subdomain Model



2.5 支付子领域模型
Payment Subdomain Model



2.5 支付子领域模型
Payment Subdomain Model

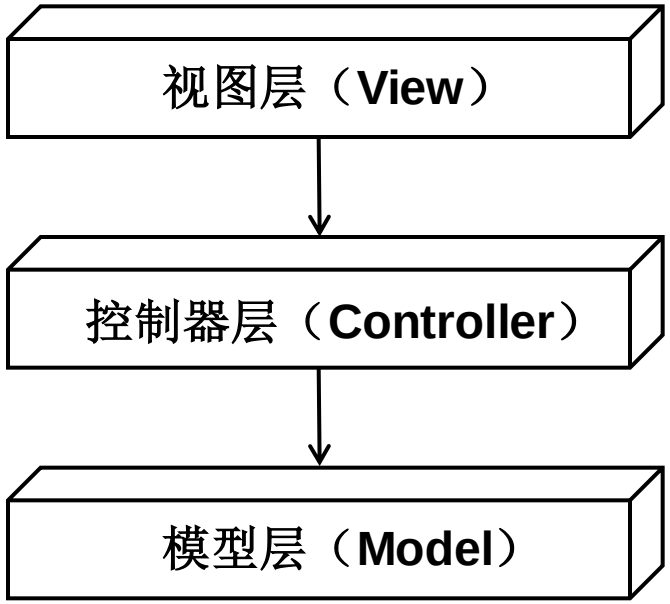
微信	支付宝	描述
appid：服务商应用ID	app_id: 支付宝分配给开发者的应用ID	
sp_mchid：服务商户号		
sub_mchid：子商户号	seller_id: 收款支付宝账号	
out_trade_no:商户订单号	out_trade_no:商户订单号	商户系统内部订单号，只能是数字、大小写字母_
amount：订单金额	total_amount:金额	单位为分
transaction_id: 微信支付订单号	trade_no:支付宝交易号	
trade_state: 交易状态	trade_status: 交易状态	
sp_openid: 支付用户openid	buyer_logon_id：买家支付宝账号	
refund_id: 支付退款单号	trade_no	
out_refund_no: 商户系统内部的退款单号	out_trade_no	
refund: 退款金额	refund_amount: 退款金额	
channel:退款渠道	fund_channel:资金渠道	
user_received_account：退款入账户	buyer_user_id：买家支付宝id	
success_time：退款成功时间	gmt_refund_pay:退款时间	
success_time：支付成功时间	send_pay_date：本次交易打款给卖家的时间	
return_mchid: 回退商户号		
out_order_no: 商户分账单号	out_request_no: 确认结算请求流水号	
out_return_no: 商户回退单号		
return_id: 微信回退单号	trade_no: 支付宝交易号	
order_id: 微信分账单号	trade_no: 支付宝交易号	

3. 体系结构

Architecture

3.1 层次体系结构
Layers Architecture

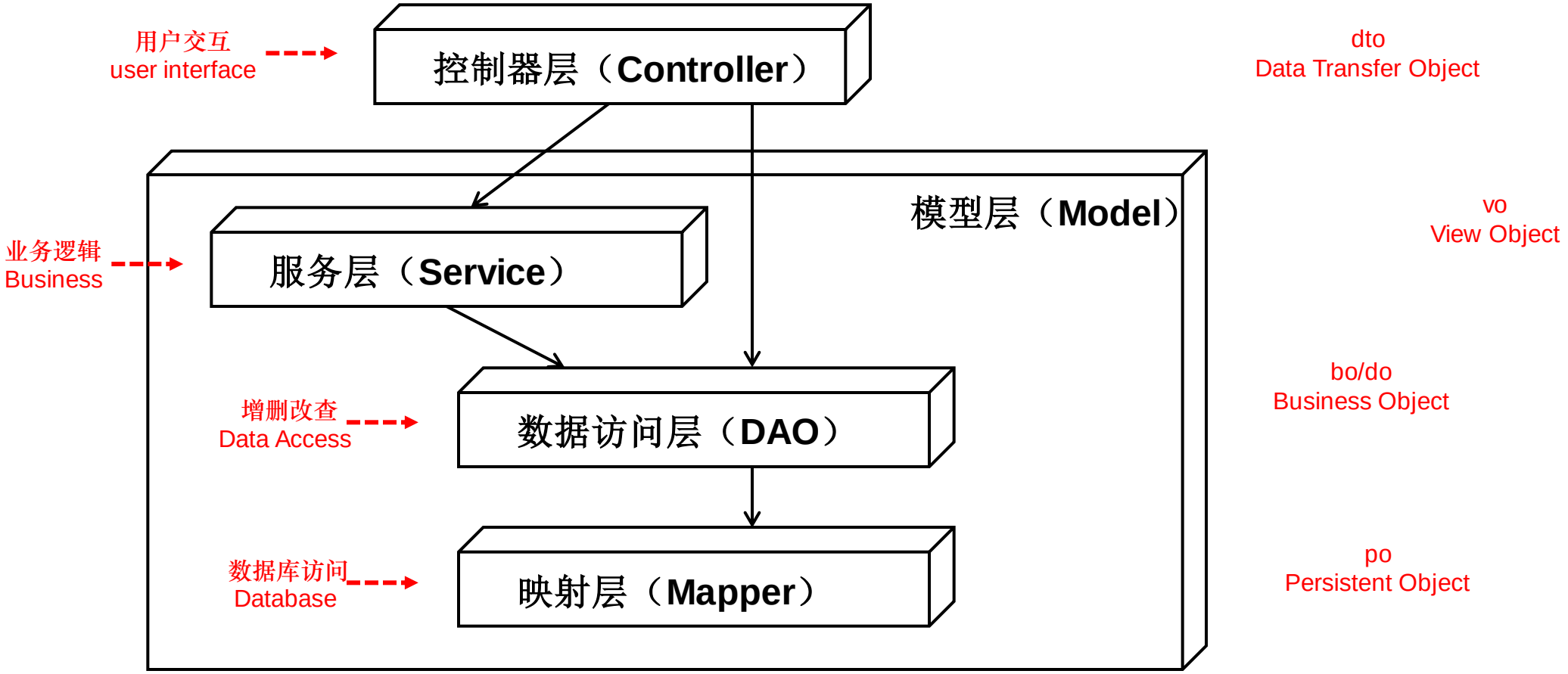
- 系统的逻辑结构
  - 大粒度的系统划分方式，每层具备清晰的职责，具备内聚性
  - 层内耦合大，层间耦合小
    - 变更控制在一层之内
    - 不同的逻辑可以分开

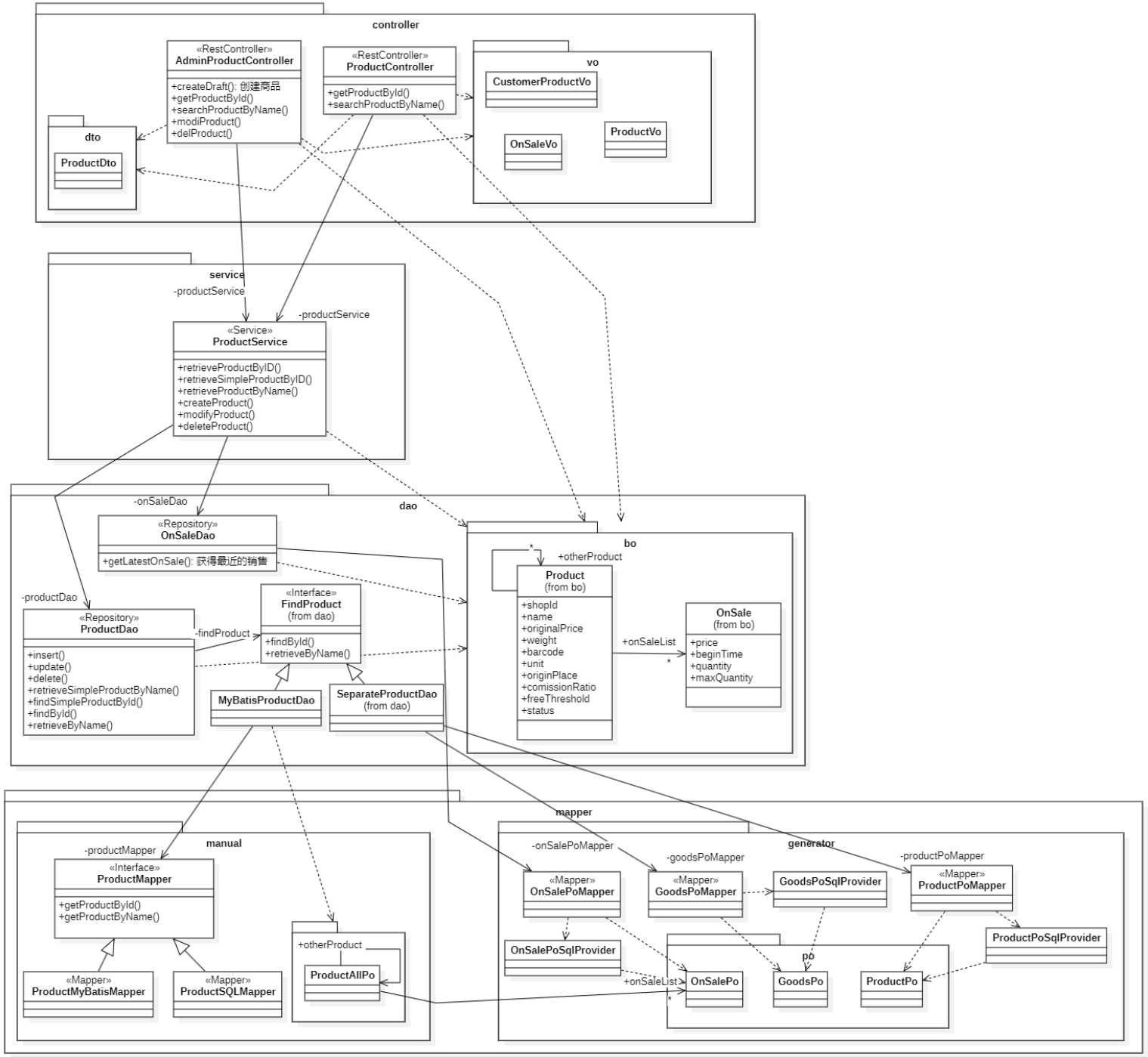


3.1 层次体系结构
Layers Architecture

- 严格层次体系结构
  - 上层代码只能调用其直接下层代码
- 松散层次体系结构
  - 上层可以调用其所有的下层代码

3.1 层次体系结构
Layers Architecture



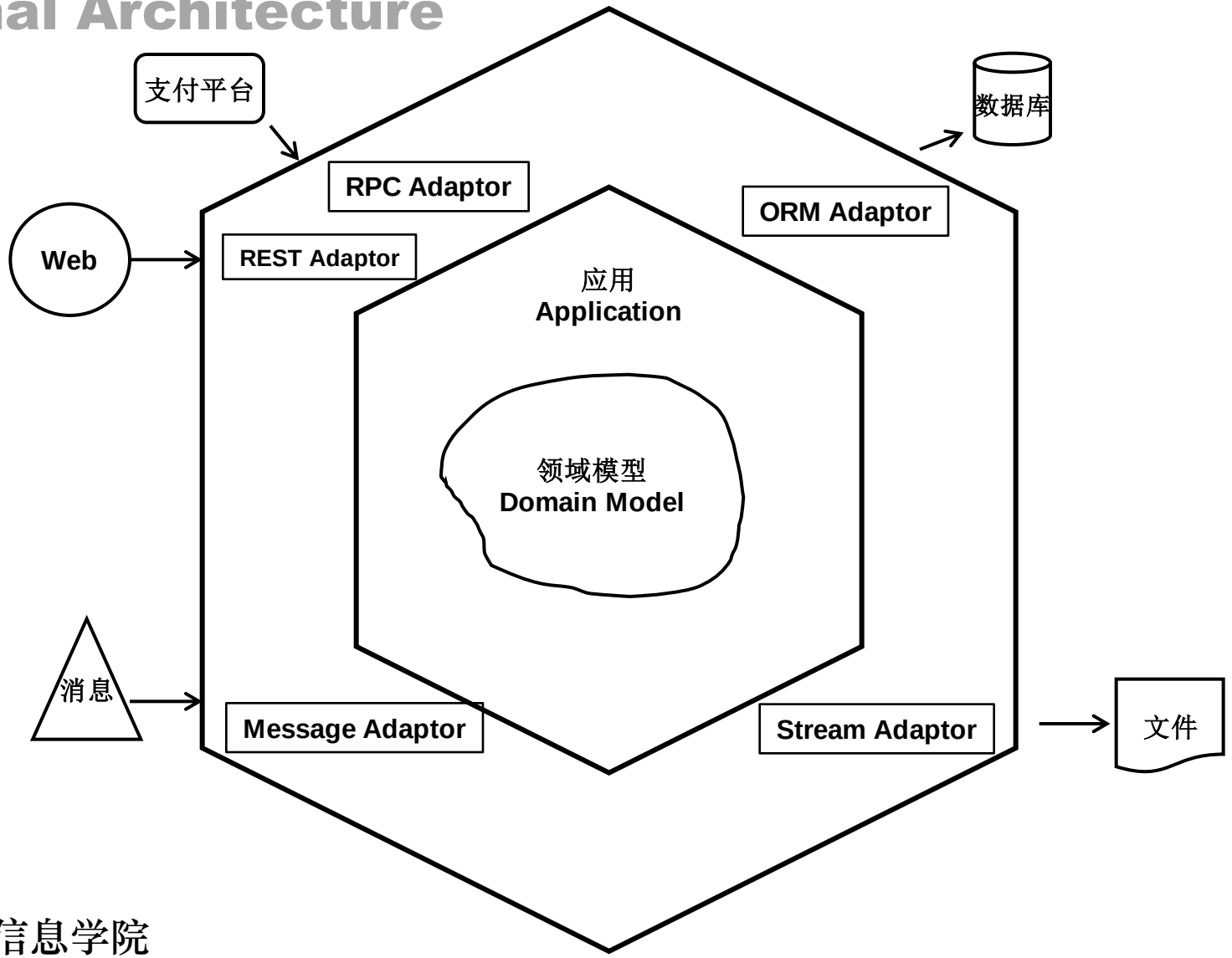

controller
«RestController»
AdminProductController
+createDraft(): 创建商品
+getProductById()
+searchProductByName()
+modiProduct()
+delProduct()
«RestController»
ProductController
+getProductById()
+searchProductByName()
vo
CustomerProductVo
OnSaleVo
ProductVo
dto
ProductDto
service
-productService
-productService
«Service»
ProductService
+retrieveProductByID()
+retrieveSimpleProductByID()
+retrieveProductByName()
+createProduct()
+modifyProduct()
+deleteProduct()
dao
-onSaleDao
«Repository»
OnSaleDao
+getLatestOnSale(): 获得最近的销售
bo
+otherProduct
Product
(from bo)
+shopId
+name
+originalPrice
+weight
+barcode
+unit
+originPlace
+comissionRatio
+freeThreshold
+status
+onSaleList
OnSale
(from bo)
+price
+beginTime
+quantity
+maxQuantity
-productDao
«Repository»
ProductDao
+insert()
+update()
+delete()
+retrieveSimpleProductByName()
+findSimpleProductById()
+findById()
+retrieveByName()
-findProduct
«Interface»
FindProduct
(from dao)
+findById()
+retrieveByName()
MyBatisProductDao
SeparateProductDao
(from dao)
mapper
manual
-productMapper
«Interface»
ProductMapper
+getProductById()
+getProductByName()
+otherProduct
ProductAllPo
«Mapper»
ProductMyBatisMapper
«Mapper»
ProductSQLMapper
generator
-onSalePoMapper
-goodsPoMapper
-productPoMapper
«Mapper»
OnSalePoMapper
«Mapper»
GoodsPoMapper
GoodsPoSqlProvider
«Mapper»
ProductPoMapper
ProductPoSqlProvider
OnSalePoSqlProvider
po
+onSaleList
OnSalePo
GoodsPo
ProductPo

3.1 层次体系结构
Layers Architecture

- PO, BO, VO和DTO
  - DTO(Data Transfer Object):前端向后端传输的数据对象，需要做合法性校验。
  - VO(View Object):后端向前端传输的用于界面展示的数据。
  - BO(Business Object):业务对象，由 Dao 层输出的封装业务逻辑的对象。
  - PO(Persistence Object):此对象与数据库表结构一一对应，通过 Mapper 层向上传输数据源对象。

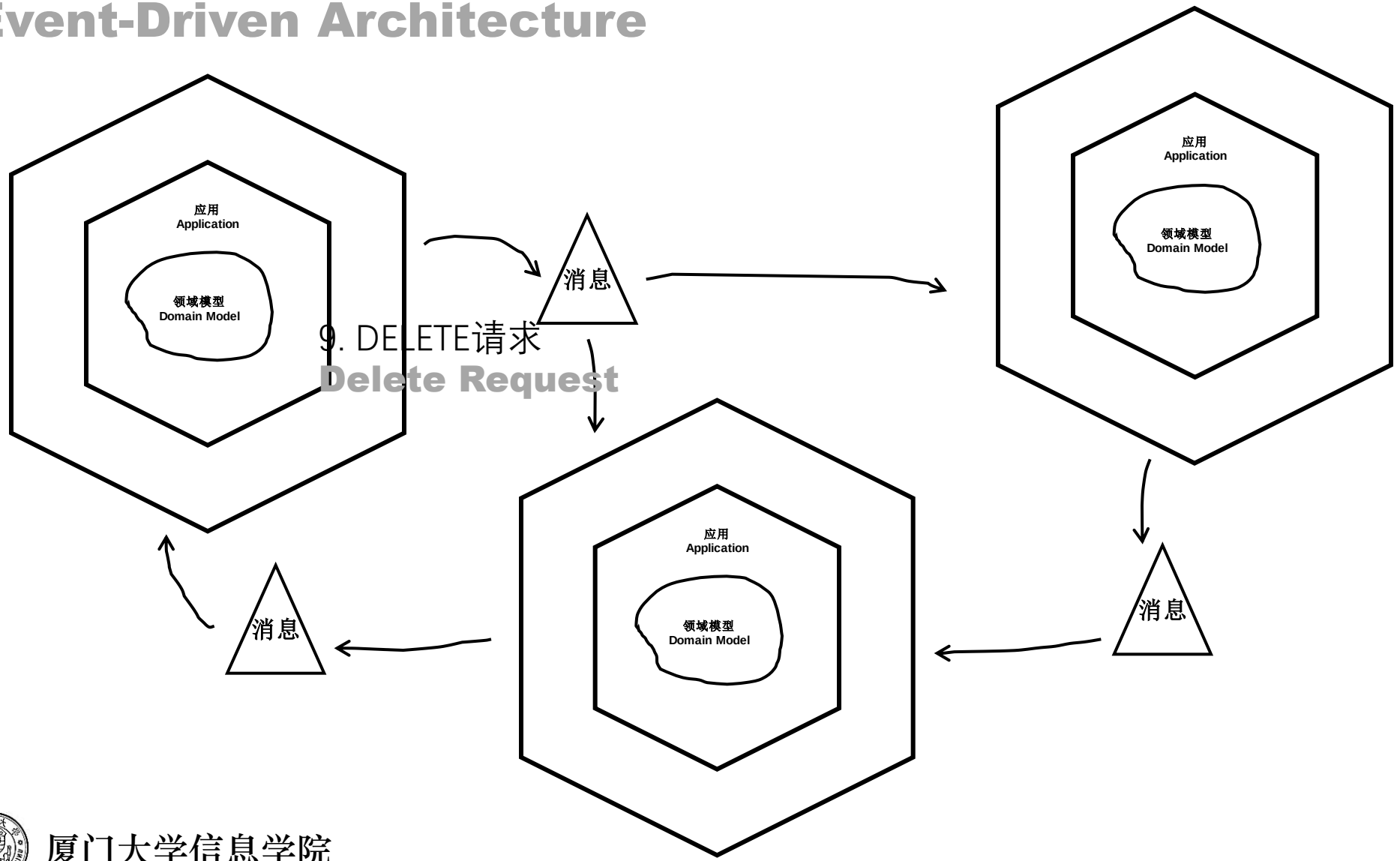
3.2 六边形体系结构

Hexagonal Architecture



3.3 消息驱动体系结构

Event-Driven Architecture

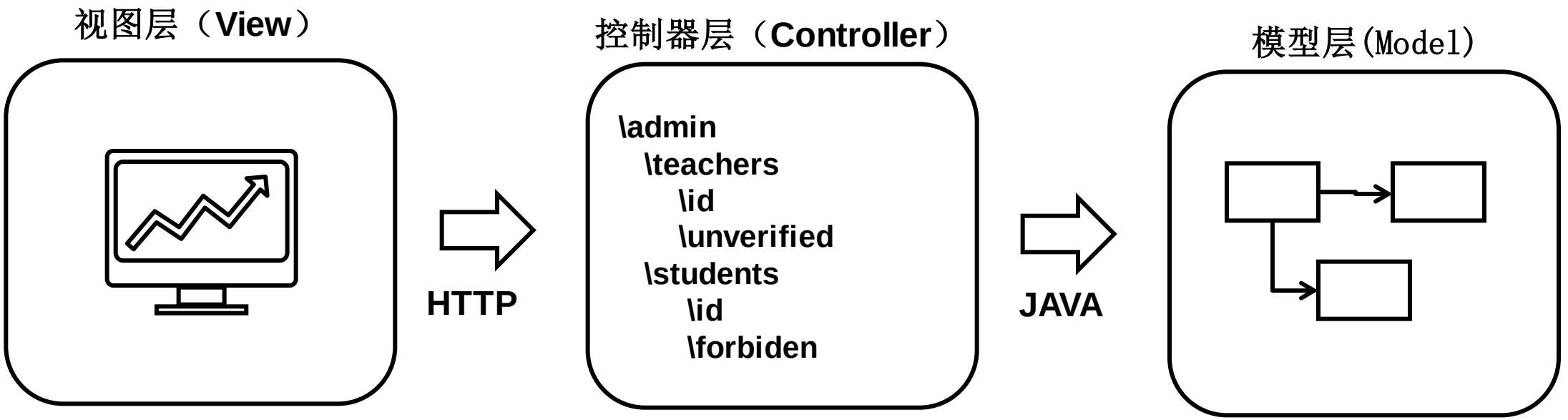


4. API 设计

API Design

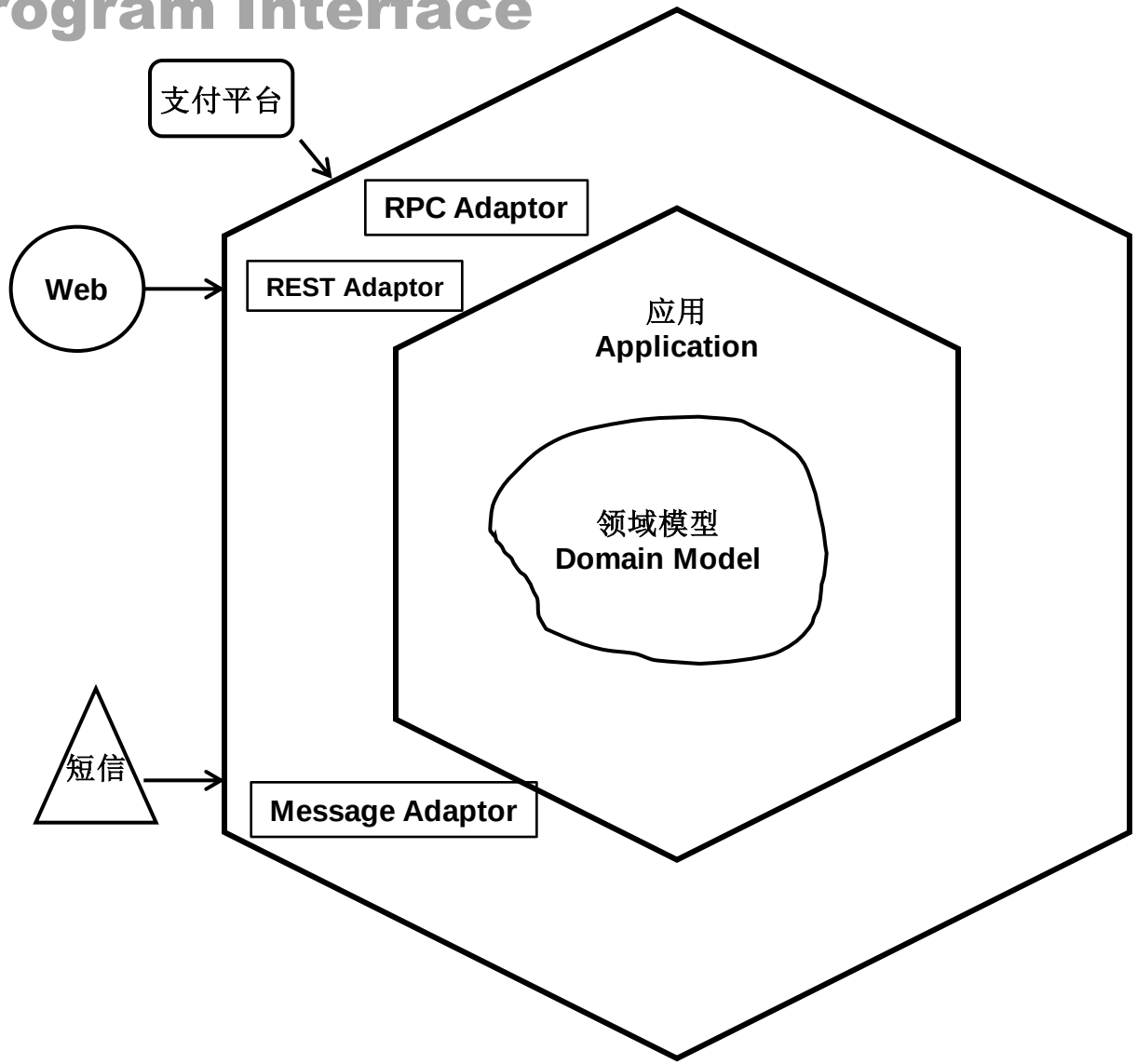
4.1 API

Application Program Interface



4.1 API

Application Program Interface

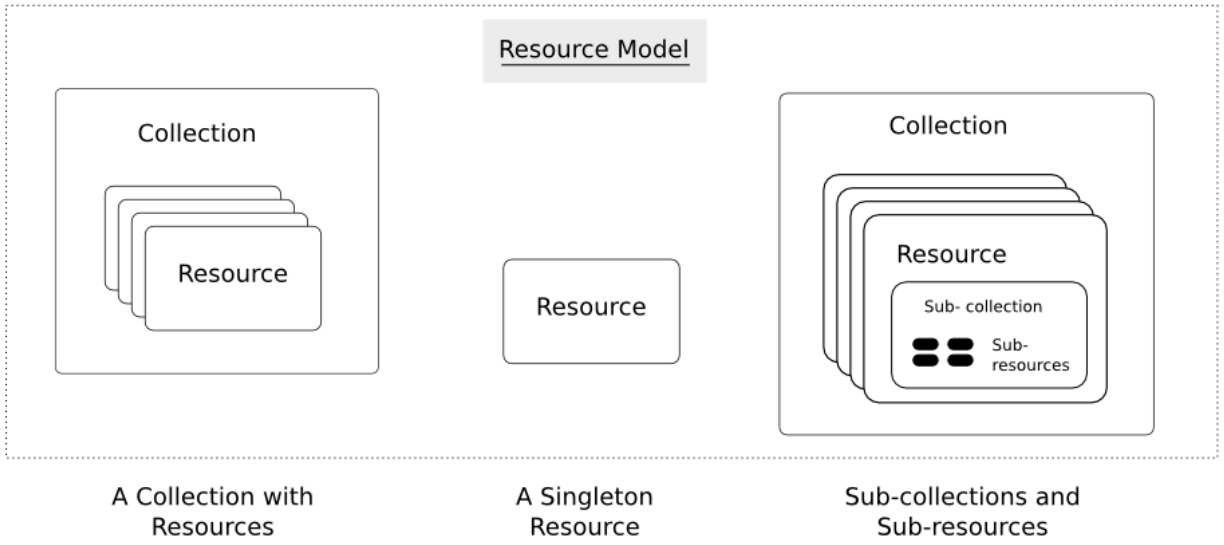


4.2 REST的基本特征
REST Essentials

- 资源（Resource）
  - 资源是应用系统想暴露给外部访问和操作的基本信息。
- 表述（Representational）
  - 可以用XML，JSON，纯文本等多种方式表述REST的资源
- 无状态通讯（Stateless Communication）
  - 客户端通过HTTP的GET/POST/PUT/DELETE对资源进行操作
  - 每一次操作都与上一次操作无关

4.3 资源
Resource

- 资源是应用系统想暴露给外部访问和操作的基本信息。



4.3 资源
Resource

- Restful API用URL表示特定的资源或资源集合

https://demo.xmu.edu.cn/oomall/api/v1/products

https://demo.xmu.edu.cn/oomall/api/v1/shops

https://demo.xmu.edu.cn/oomall/api/v1/categories

4.3 资源
Resource

- URL构成

URL	描述
/categories	商品的所有一级分类categories
/categories/1	id为1的一级分类
/categories/1/subcategories	id为1的一级分类下的所有二级分类
/subcategories/2	id为2的二级分类
/subcategories/2/products	id为2的二级分类下所有商品
/categories/1/products	id为1的一级分类下所有商品
/products/5	id为5的商品

4.4 表述
Representation

- JSON（JavaScript Object Notation）
  - 是一种轻量级的数据交换格式。它采用完全独立于编程语言的文本格式来存储和表示数据。JSON易于人阅读和编写的同时，便于机器解析和生成，可有效地提升网络传输效率，是理想的数据交换语言。

4.4 表述
Representation

- JSON的构成
  - 值可以是对象、数组、数字、字符串或者三个字面值(false、null、true)中的一个。

对象：

```
{"name": "John Doe", "age": 18, "address": {"country" : "china", "zip-code": "10000"}}
```

数组：

```
[3, 1, 4, 1, 5, 9, 2, 6]
```

4.5 无状态通讯
Stateless Communication

- 利用HTTP的四种请求方法

请求方法	用途
POST	创建一个新的资源并返回新资源
GET	查询特定资源，可重复操作，无副作用
PUT	修改资源的全部属性并返回修改后的资源
DELETE	删除资源

4.5 无状态通讯
Stateless Communication

- HTTP状态码

HTTP状态码	描述
2**	操作成功接收并处理
3**	重定向
4**	客户端错误，请求包含语法错误
5**	服务器错误，服务器在处理请求的过程中发生了错误

4.5 无状态通讯
Stateless Communication

- 所有请求共用的HTTP状态码

共用HTTP状态码	描述
400	参数格式错误（缺少或不符合要求）
401	用户需登录访问
403	用户无权限访问
409	资源冲突或资源被锁定（由于事务回滚造成的）
500	服务器通用错误（服务器程序错误）
503	服务当前无法处理请求（服务器忙）

4.6 GET 请求
GET Request

URL	描述
GET /categories	获取所有的一级商品分类
GET /categories/12	获取id为12的一级商品分类对象
GET /categories/12/products	获取id为12的一级商品分类下所有商品
GET /categories/12/subcategories	获取id为12的分类下的所有子分类
GET /products?name=电视机&sort=price&order=acend	获取商品名称为电视机的商品，按照价格升序排序
GET /hot-products?page=2&pagesize=10	以分页方式获取所有秒杀的商品中的第二页，每页10个商品

4.6 GET 请求
GET Request

- Get请求的HTTP状态码

Get请求状态码	描述
200	请求成功，数据在Response中返回

4.7 POST请求
POST Request

URL	描述
POST /categories	创建一个新的一级商品分类
POST /categories/12/subcategories	在id为12的一级商品分类下创建一个二级分类
POST /subcategories/19/products/1	在id为19的二级商品分类下增加id为1的商品
POST /orders/1000/payments	为id为1000的订单付款
POST /login	用户登录

4.7 POST请求
POST Request

- Post请求的HTTP状态码

HTTP状态码	描述
201	新资源成功，数据在Response中返回
202	已经接受处理请求但尚未完成（异步处理）

4.8 PUT 请求

PUT Request

URL	描述
PUT /categories/12	修改id为12的一级商品分类
PUT /subcategories/13	修改id为13的二级商品分类
PUT /products/5/onshelves	将id为5的商品上架
PUT /products/5/offshelves	将id为5的商品下架

4.8 PUT 请求
PUT Request

- PUT请求的HTTP状态码

HTTP状态码	描述
200	修改成功
202	已经接受处理请求但尚未完成（异步处理）

4.9 DELETE请求
DELETE Request

URL	描述
DELETE /categories/12	删除id为12的一级商品分类
DELETE /categories/12/products/5	在id为12的分类中将id为5的商品移除

4.9 DELETE请求
DELETE Request

- Delete请求的HTTP状态码

HTTP状态码	描述
200	删除成功
202	已经接受处理请求但尚未完成（异步处理）

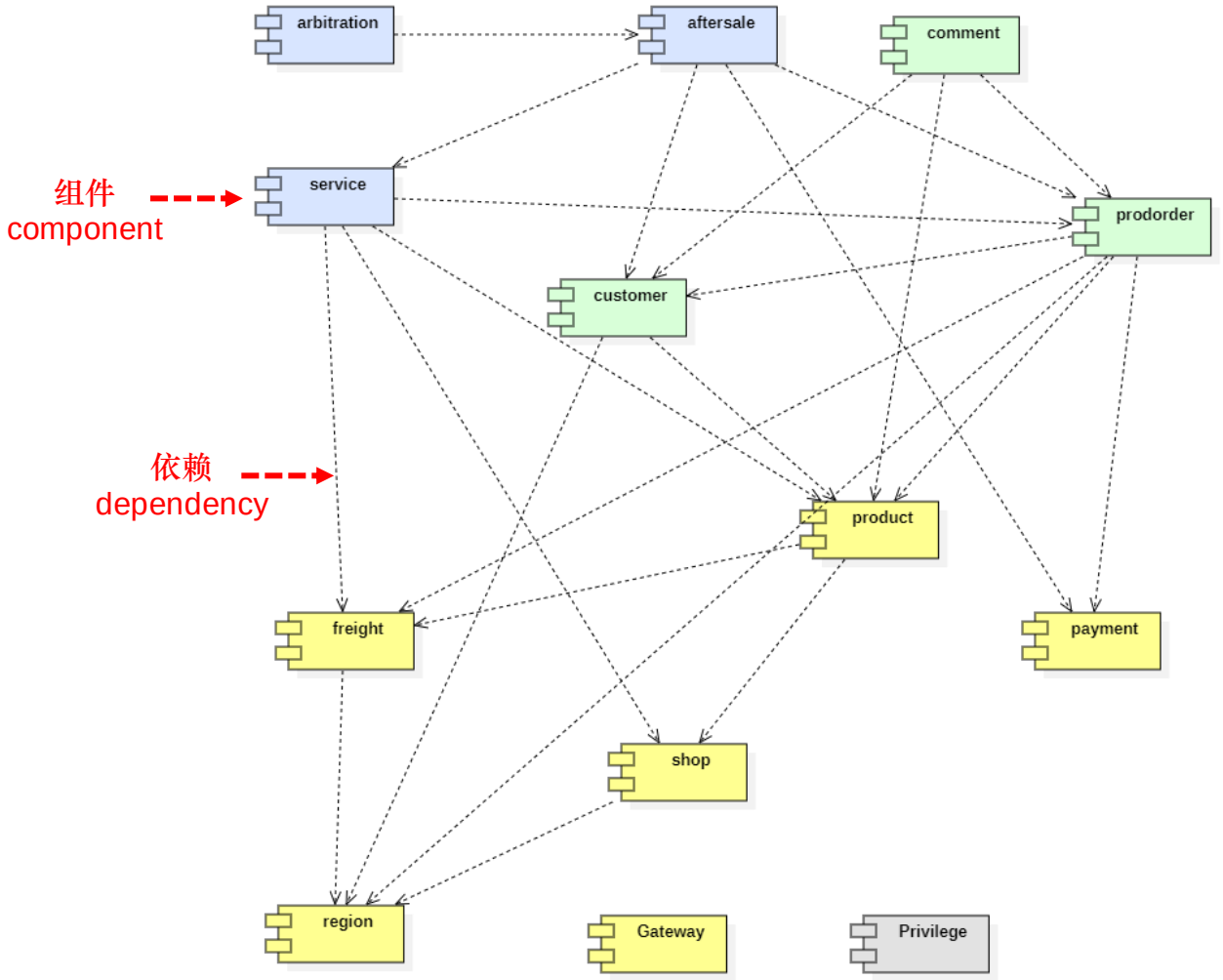
5. 组件图与部署图

Component Diagram and Deployment Diagram

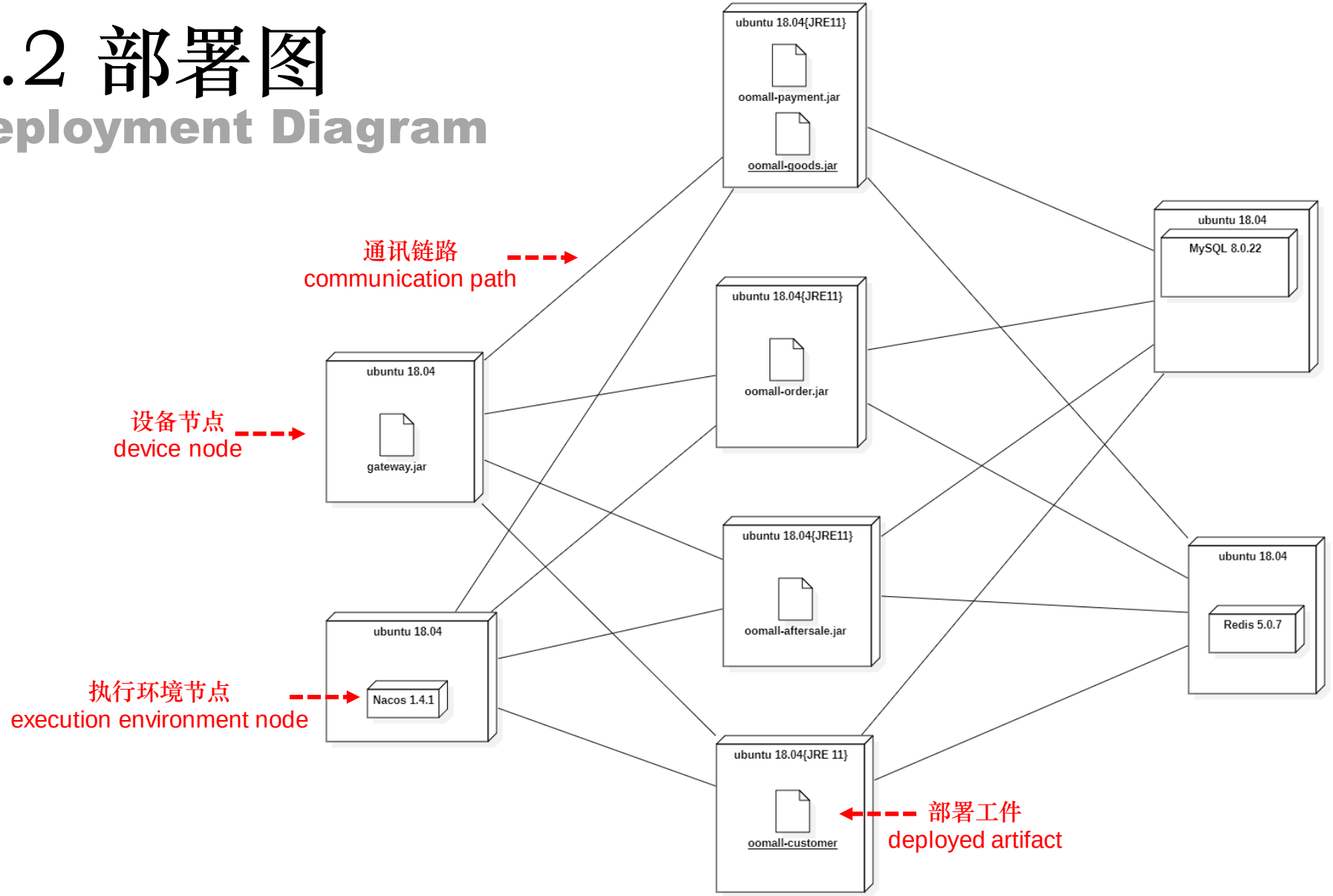
5.1 组件图
Component Diagram

- 组件是软件系统的模块
  - 包含了特定的逻辑，并在执行环境中是可替换的
- UML组件强调以下两点
  - 组件是通过接口被调用的
  - 模块化、自包含、可替换

5.1 组件图
Component Diagram



5.2 部署图
Deployment Diagram



5.2 部署图
Deployment Diagram

